UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Software

Aplicaciones Distribuidas (ISR-701)

FORM-FB-01

Fichas bibliográficas: fundamentos de sistemas distribuidos y arquitecturas distribuidas

Grupo: CLM
Integrantes: Castro Coello Mario Sebastián
Loor Suárez Jordan Josué
Mendoza Villarroel Elian Jossue
Docente: Gleiston Cicerón Guerrero Ulloa, PhD
Nivel / paralelo: Séptimo nivel / B
Periodo: SPA 2026–2027
Fecha: 8 de noviembre de 2026

Repositorio: `https://github.com/jloors13/form-fb-01-fichas-bibliograficas`

Índice

1. Fichas de Castro Coello Mario Sebastián **2**
1.1. Ficha 1 . . . 2
1.2. Ficha 2 . . . 2
1.3. Ficha 3 . . . 3
1.4. Ficha 4 . . . 3
1.5. Ficha 5 . . . 4

2. Fichas de Loor Suárez Jordan Josué **5**
2.1. Ficha 1: Seguridad en microservicios . . . 5
2.2. Ficha 2: Evolución de API en microservicios . . . 5
2.3. Ficha 3: Mensajería descentralizada y protección de contenido privado . . . 6
2.4. Ficha 4: Reenvío de mensajes y difusión de contenido en WhatsApp . . . 7
2.5. Ficha 5: Comunicación y rendimiento en microservicios . . . 7

3. Fichas de Mendoza Villarroel Elian Jossue **8**
3.1. Ficha 1: Identificación de microservicios al descomponer un monolito . . . 8
3.2. Ficha 2: Descomposición de monolitos en microservicios . . . 9
3.3. Ficha 3: Prácticas de seguridad en sistemas de microservicios . . . 9
3.4. Ficha 4: Herramientas de monitoreo para DevOps y microservicios . . . 10
3.5. Ficha 5: Visualización dinámica de arquitecturas de microservicios . . . 10

4. Declaración de uso de inteligencia artificial **12**
4.1. Castro Coello Mario Sebastián . . . 12
4.2. Loor Suárez Jordan Josué . . . 13
4.3. Mendoza Villarroel Elian Jossue . . . 13

Referencias **14**

1. Fichas de Castro Coello Mario Sebastián

1.1. Ficha 1

Referencia (IEEE):

V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,» *IEEE Access*, vol. 11, págs. 88 339-88 358, 2023. DOI: 10.1109/ACCESS.2023.3305687

DOI o ISBN verificado:

https://doi.org/10.1109/ACCESS.2023.3305687 (verificado el 05/11/2026)

Resumen:

El artículo parte de un vacío: hay bastante trabajo sobre cómo migrar monolitos a microservicios, pero poco sobre los principios que deberían guiar ese diseño. Los autores, de la Escuela Politécnica Nacional, hacen una revisión de literatura. Repasan principios de la orientación a objetos, como el encapsulamiento y el ocultamiento de información, y los relacionan con ambas arquitecturas. Presentan una lista de patrones usados en microservicios y comparan cómo Martin Fowler y Sam Newman proponen descomponer un sistema. Relacionan esas propuestas con el trabajo de David Parnas sobre modularidad. Cierran con ventajas, desventajas y tendencias actuales de cada una.

Dato concreto:

Los autores parten de que hay un vacío en el conocimiento sobre los principios que se deben adoptar al implementar microservicios, aunque sí existen trabajos sobre el proceso de migración. También recogen la idea de Parnas de que modularizar un sistema mejora su flexibilidad y lo hace más fácil de entender (Abstract, p. 88339).

Nota personal:

Me ayuda a pensar los límites de un servicio igual que los de un módulo: cada uno debe esconder una decisión de diseño. La usaré en TA-IND-01, en el análisis de la arquitectura de Discord, para evaluar cómo reparte responsabilidades. Un caso concreto: Discord puso servicios de datos en Rust entre su API y ScyllaDB, sin lógica de negocio, que se encargan de agrupar consultas repetidas a la misma fila. La API no necesita saber cómo se hace esa agrupación, y eso se puede analizar como ocultamiento de información en el sentido de Parnas.

1.2. Ficha 2

Referencia (IEEE):

S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Benchmarking Consistency Levels of Cloud-Distributed NoSQL Databases Using YCSB,» *IEEE Access*, vol. 13, págs. 63 428-63 438, 2025. DOI: 10.1109/ACCESS.2025.3558923

DOI o ISBN verificado:

https://doi.org/10.1109/ACCESS.2025.3558923 (verificado el 05/11/2026)

Resumen:

El artículo estudia cuánto cuesta la consistencia fuerte en bases de datos NoSQL replicadas en varias regiones de la nube. Mantener todos los nodos con el dato más reciente choca con la disponibilidad y el rendimiento. Los autores probaron Cassandra, MongoDB y Redis, que usan modelos de datos distintos: columnas anchas, documentos y clave-valor. Usaron YCSB para generar cargas de trabajo estándar y compararon configuraciones de consistencia débil y fuerte. Para la fuerte, en MongoDB combinaron linearizable con w:n, y en Redis usaron

el comando WAIT. Los resultados muestran caídas grandes de rendimiento con consistencia fuerte en Cassandra y Redis.

Dato concreto:

En Cassandra, las operaciones de lectura y escritura por segundo bajaron hasta 95 % en algunas cargas de trabajo. En Redis, la consistencia fuerte hizo las operaciones más de 20 veces más lentas (Abstract, p. 63428).

Nota personal:

El artículo mide el mismo intercambio que Discord tuvo que resolver con sus mensajes. Discord pasó sus mensajes de MongoDB a Cassandra antes de 2017 y luego los migró a ScyllaDB, que es compatible con Cassandra, pasando de 177 a 72 nodos. Lee y escribe con nivel de consistencia quorum, y su propio equipo cuenta que, por eso, una partición caliente subía la latencia de todas las consultas a esos nodos. La usaré en TA-IND-01, en el análisis del almacenamiento y la consistencia de datos, para respaldar con cifras que la consistencia elegida tiene un costo de rendimiento medible.

1.3. Ficha 3

Referencia (IEEE):

A. U. Rehman, R. L. Aguiar y J. P. Barraca, «Fault-Tolerance in the Scope of Cloud Computing,» *IEEE Access*, vol. 10, págs. 63 422-63 441, 2022. DOI: 10.1109/ACCESS.2022.3182211

DOI o ISBN verificado:

https://doi.org/10.1109/ACCESS.2022.3182211 (verificado el 05/11/2026)

Resumen:

El artículo revisa cómo mantener servicios disponibles y confiables en la nube cuando fallan el hardware, el software o la red. Es una revisión de más de 100 estudios. Primero explica la nube y sus modelos de servicio y de despliegue. Luego distingue falla, error y fallo, clasifica los tipos de falla y presenta métricas como MTTF, MTTR, MTBF y disponibilidad. Después separa las técnicas reactivas, como checkpoint, replicación y reintento, de las proactivas, como self-healing, rejuvenecimiento de software y migración preventiva. Concluye que combinar los dos enfoques da mejores resultados, aunque lo proactivo agrega costo de monitoreo.

Dato concreto:

El artículo menciona la meta de disponibilidad «Five 9's» (es decir, 99,999 %), que permite 5 min y 15 segundos de caída al año (Sección IV.B, «Needs and Applications for Fault-Tolerance Computing», p. 63427). Las definiciones de MTTF, MTTR y MTBF están en la Sección IV.C, p. 63428.

Nota personal:

Me da un vocabulario preciso: separa falla, error y fallo, y divide las técnicas en reactivas y proactivas. La usaré en TA-IND-01, en el análisis de tolerancia a fallos, para clasificar los mecanismos de Discord con esas categorías. Por ejemplo, Discord replica cada partición de mensajes en tres nodos, y el artículo ubica la replicación entre las técnicas reactivas. Las métricas de disponibilidad me sirven para describir los requisitos que Discord se puso al elegir base de datos, como el failover automático.

1.4. Ficha 4

Referencia (IEEE):

W. Cazzola, F. Cesarini y L. Tansini, «PerformERL: a performance testing framework for

erlang,» *Distributed Computing*, vol. 35, n.º 5, págs. 439-454, 2022. DOI: 10.1007/s00446-022-00429-7

DOI o ISBN verificado:
https://doi.org/10.1007/s00446-022-00429-7 (verificado el 05/11/2026)

Resumen:
El artículo parte de un problema en sistemas Erlang: los fallos de rendimiento suelen aparecer recién en producción. Las herramientas de carga miden el sistema desde afuera y las de monitoreo no generan una carga repetible. Los autores proponen PerformERL, un framework que junta ambas cosas. Inyecta tres agentes en cada nodo: uno cuenta llamadas y tiempos, otro descubre los procesos involucrados y otro recoge memoria y reducciones. El usuario define la carga en un módulo de prueba y recibe un reporte HTML. Lo evaluaron midiendo memoria y overhead, y comparándolo con eprof, fprof y Wombat sobre el servidor cowboy.

Dato concreto:
Sobre el servidor HTTP cowboy, PerformERL hizo las respuestas unas 6 veces más lentas al inicio y se estabilizó cerca de 2 veces bajo carga. Con eprof la ralentización fue de 1,4 veces y con fprof de 5 veces (Sección 5.3, «PerformERL in the real world», Fig. 6). Los tres agentes se definen en las Secciones 3.3, 3.4 y 3.5.

Nota personal:
Discord corre sobre la máquina virtual de Erlang: su sistema en tiempo real está escrito en Elixir, y cada sesión de usuario y cada servidor (guild) es un proceso. El artículo me ayuda a entender qué se puede medir dentro de ese tipo de sistema: tiempo de llamadas, memoria y cola de mensajes por proceso. La usaré en TA-IND-01, en el análisis de rendimiento de la capa en tiempo real, para explicar el problema que Discord reportó en servidores con hasta 30 000 usuarios concurrentes: los procesos de guild no alcanzaban a vaciar sus colas de mensajes en horas pico. No voy a ejecutar PerformERL. Lo uso para explicar qué métricas internas dejan ver ese cuello de botella.

1.5. Ficha 5

Referencia (IEEE):
G. Blinowski, A. Ojdowska y A. Przybylek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

DOI o ISBN verificado:
https://doi.org/10.1109/ACCESS.2022.3152803 (verificado el 05/11/2026)

Resumen:
El artículo cuestiona si migrar a microservicios conviene en sistemas pequeños, sin miles de usuarios concurrentes, porque la evidencia viene sobre todo de Netflix o Amazon. Los autores construyeron una aplicación con dos servicios REST, uno liviano y otro de cálculo intensivo, en cuatro versiones: monolito y microservicios, en Java y en C# .NET. Midieron el throughput con JMeter en un PC con Windows 10 Enterprise, Azure Spring Cloud y Azure App Service. En una sola máquina el monolito rindió más. En la nube, escalar verticalmente fue más rentable, y pasar de cierto número de instancias empeoró el rendimiento.

Dato concreto:
En la máquina local, el monolito en .NET atendió en promedio más del doble de solicitudes que su versión en microservicios en el servicio City (Sección V.A, «Local Environment», pp.

20364–20365). En Azure App Service, el throughput del servicio Route bajó al subir a 20 instancias (Sección V.C, «Azure App Service Environment», p. 20368).

Nota personal:

El estudio se limita a sistemas sin miles de usuarios concurrentes que pueden escalar verticalmente. Discord está en el caso opuesto: en 2017 ya tenía cerca de cinco millones de usuarios concurrentes, así que la conclusión a favor del monolito no se le aplica. Lo que sí me sirve es el otro hallazgo: escalar horizontalmente tiene un límite, porque el balanceo de carga agrega su propio costo. La usaré en TA-IND-01, en el análisis de escalabilidad, para explicar por qué Discord necesita escalar horizontalmente y por qué eso tiene un precio. También me sirve para un matiz: el equipo de Discord llama «API monolith» a su API, así que el sistema no está dividido en microservicios por completo.

2. Fichas de Loor Suárez Jordan Josué

2.1. Ficha 1: Seguridad en microservicios

Referencia (IEEE):

R. Kanishka Jayalath, H. Ahmad, D. Goel, M. Shuja Syed y F. Ullah, «Microservice Vulnerability Analysis: A Literature Review With Empirical Insights,» *IEEE Access*, vol. 12, págs. 155 168-155 204, 2024. DOI: 10.1109/ACCESS.2024.3481374

DOI o ISBN verificado:

https://doi.org/10.1109/ACCESS.2024.3481374

Resumen:

Este artículo aborda los problemas de seguridad que se pueden presentar en las arquitecturas de microservicios, particularmente por la comunicación entre servicios y dependencias que existen entre sus componentes. Para analizar esta situación, los autores realizaron una revisión de 62 estudios académicos mediante la cual identificaron 126 vulnerabilidades y las organizaron en una clasificación. De igual manera, evaluaron cuatro aplicaciones de referencia utilizando tres herramientas de análisis de seguridad. Durante las pruebas detectaron 1667 vulnerabilidades. Los hallazgos destacan la importancia de utilizar diferentes herramientas de detección y mantener una supervisión continua para identificar riesgos y fortalecer la protección de los sistemas basados en microservicios.

Dato concreto:

Los investigadores detectaron 1667 vulnerabilidades durante el análisis de cuatro aplicaciones de microservicios utilizando tres herramientas de escaneo.

Ubicación: sección V, página 155193.

Nota personal:

Esta investigación me permitió comprender que la arquitectura de microservicios puede presentar diferentes riesgos de seguridad debido a la comunicación y las dependencias entre sus componentes. En TA-IND-01 utilizaré estos conocimientos para analizar los posibles riesgos y las medidas de protección que deben considerarse en un sistema distribuido como WhatsApp.

2.2. Ficha 2: Evolución de API en microservicios

Referencia (IEEE):

A. Lercher, J. Glock, C. Macho y M. Pinzger, «Microservice API Evolution in Practice: A Study on Strategies and Challenges,» *Journal of Systems and Software*, vol. 215, 2024, **artno** 112110. DOI: 10.1016/j.jss.2024.112110

DOI o ISBN verificado:

https://doi.org/10.1016/j.jss.2024.112110

Resumen:

El artículo estudia los desafíos que surgen cuando se modifican las interfaces API utilizadas por los microservicios para comunicarse entre sí. Estas modificaciones pueden generar incompatibilidades y afectar a otros servicios que dependen de ellas. Para abordar el problema, los autores realizaron 17 entrevistas semiestructuradas a desarrolladores, arquitectos y responsables de 11 empresas. Por medio del análisis de las entrevistas, identificaron seis estrategias y seis desafíos relacionados con la evolución de las API. Entre sus resultados destaca la importancia de mantener la compatibilidad con versiones anteriores, coordinar los cambios entre equipos y mejorar la comunicación para evitar errores e interrupciones.

Dato concreto:

En promedio, las API REST representaban el 66,8 % de la comunicación total de las organizaciones participantes.

Ubicación: página 3, sección 3.1.

Nota personal:

Esta investigación me permitió comprender que las modificaciones en las interfaces API pueden afectar la comunicación entre los servicios que dependen de ellas. En TA-IND-01 utilizaré estos conocimientos para analizar la importancia de la comunicación y la compatibilidad entre los componentes de un sistema distribuido como WhatsApp.

2.3. Ficha 3: Mensajería descentralizada y protección de contenido privado

Referencia (IEEE):

M. Franco, O. Gaggi, B. Guidi, A. Michienzi y C. E. Palazzi, «A Decentralised Messaging System Robust Against the Unauthorised Forwarding of Private Content,» *Future Generation Computer Systems*, vol. 145, págs. 211-222, 2023. DOI: 10.1016/j.future.2023.03.025

DOI o ISBN verificado:

https://doi.org/10.1016/j.future.2023.03.025

Resumen:

En este artículo se investiga cómo se tiene que evitar que las personas compartan imágenes privadas de otras personas sin su autorización por medio de aplicaciones de mensajería. Los investigadores primero evaluaron diez plataformas para identificar las características de seguridad y privacidad que poseen, más adelante se diseñaron dos alternativas descentralizadas que facilitan reconocer imágenes mediante los valores hash guardados en los dispositivos de cada persona. Para comprobar su desempeño se realizaron simulaciones con diferentes cantidades de participantes y verificaron los tiempos de procesamiento y comunicación frente a una arquitectura centralizada.

Los resultados mostraron ventajas en los tiempos de respuesta y en la distribución del procesamiento.

Dato concreto:

En las pruebas realizadas con 10.000 nodos, la arquitectura centralizada superó los 90 milisegundos de procesamiento por imagen, mientras que las alternativas descentralizadas necesitaron solamente unos pocos milisegundos en promedio.

Ubicación: página 219, sección 7.5, Scalability Evaluation.

Nota personal:

La investigación permite comprender que las imágenes privadas son importantes de proteger, y de cómo distribuir el procesamiento entre los dispositivos se puede reducir la dependencia de un servidor central. En TA-IND-01 aplicaré estos conocimientos para analizar los mecanismos de privacidad y de la seguridad de WhatsApp, así también las limitaciones que puedan existir al compartir contenido entre las personas.

2.4. Ficha 4: Reenvío de mensajes y difusión de contenido en WhatsApp

Referencia (IEEE):

P. d. F. Melo, M. Hoseini, S. Zannettou y F. Benevenuto, «Don't Break the Chain: Measuring Message Forwarding on WhatsApp,» en *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 18, 2024, págs. 1054-1067. DOI: 10.1609/icwsm.v18i1.31372

DOI o ISBN verificado:

https://doi.org/10.1609/icwsm.v18i1.31372

Resumen:

El artículo analiza la difusión de mensajes a través de la opción para reenviar mensajes de WhatsApp y las complicaciones que presenta la plataforma para controlar la circulación de contenido viral. Para esta investigación los autores recopilaron cerca de 9.8 millones de mensajes de 1.101 grupos públicos de Brasil vinculados con discusiones políticas. A su vez, estudiaron los tipos de contenido compartido, las cadenas de reenvío y la identificación de archivos multimedia duplicados. Los resultados mostraron que los usuarios pueden evitar las restricciones de reenvío mediante métodos como copiar y volver a enviar los archivos. Además, encontraron que una parte importante del contenido viral no recibía la etiqueta de aviso correspondiente.

Dato concreto:

Los investigadores encontraron que 612.855 apariciones de contenido multimedia duplicado, equivalentes al 59 %, no recibieron la etiqueta "Forwarded Many Times" de WhatsApp.

Ubicación: página 1061, apartado "Discrepancy between forwards and duplicates".

Nota personal:

Esta investigación me facilitó la comprensión de que las restricciones al reenvío de WhatsApp pueden presentar inconvenientes cuando las personas utilizan otras formas de compartir el mismo contenido. En TA-IND-01 utilizaré estos conocimientos para analizar cómo se distribuyen los mensajes dentro de WhatsApp y qué mecanismos utiliza la plataforma para controlar su propagación.

2.5. Ficha 5: Comunicación y rendimiento en microservicios

Referencia (IEEE):

C. Colarusso, A. De Caro, I. Falco, L. Goglia y E. Zimeo, «A distributed tracing pipeline for improving locality awareness of microservices applications,» *Software: Practice and Experience*, vol. 54, n.º 6, págs. 1118-1140, 2024. DOI: 10.1002/spe.3317

DOI o ISBN verificado:

https://doi.org/10.1002/spe.3317

Resumen:

El artículo investiga los inconvenientes que pueden surgir cuando los microservicios de una aplicación se comunican desde diferentes dispositivos, aumentando los tiempos de respuesta. Para estudiar este problema, los autores desarrollaron un sistema que recoge información

sobre las comunicaciones entre los servicios y aplica estrategias de agrupamiento para establecer cuáles deberían ejecutarse juntos. La propuesta fue analizada con TrainTicket, que es una aplicación formada por 41 microservicios y 22 bases de datos, mediante pruebas en distintas condiciones de red. Los resultados mostraron que en una distribución apoyada en las relaciones entre servicios puede disminuir los tiempos de respuesta hasta un 61 %.

Dato concreto:

Los autores obtuvieron una reducción de hasta el 61 % en los tiempos de respuesta mediante estrategias de agrupamiento basadas en la localidad.

Ubicación: página 1118, resumen del artículo.

Nota personal:

Esta investigación me permitió comprender que la ubicación de los microservicios y la comunicación entre sus componentes, puede incidir en el rendimiento de una aplicación. En TA-IND-01 usaré esta investigación para evaluar la importancia de la comunicación entre los componentes de WhatsApp y cómo la distribución de sus servicios puede influir en los tiempos de respuesta.

3. Fichas de Mendoza Villarroel Elian Jossue

3.1. Ficha 1: Identificación de microservicios al descomponer un monolito

Referencia (IEEE):

I. Oumoussa y R. Saidi, «Evolution of Microservices Identification in Monolith Decomposition: A Systematic Review,» *IEEE Access*, vol. 12, págs. 23 389-23 405, 2024, ISSN: 2169-3536. DOI: 10.1109/access.2024.3365079 dirección: http://dx.doi.org/10.1109/ACCESS.2024.3365079

DOI o ISBN verificado:

https://doi.org/10.1109/ACCESS.2024.3365079

Resumen:

El artículo aborda la transformación de sistemas monolíticos hacia arquitecturas de microservicios (MSA), que ofrece ventajas como escalabilidad, agilidad y adopción en la nube. Aun así, el éxito de esta depende de una correcta y eficiente identificación de los microservicios, área que permanece aún en desarrollo. Los autores realizaron una revisión sistemática de la literatura y seleccionaron 168 artículos para realizar este análisis. Estos hallazgos ponen de manifiesto la necesidad de herramientas más automatizadas y precisas, criterios estandarizados para su evaluación y mayor entendimiento de factores humanos. Las métricas de evaluación no se aplican de manera consistente en todos los estudios, es difícil comparar la efectividad de los métodos entre estudios.

Dato concreto:

Los autores identificaron 5 objetivos de investigación en el campo de identificación de microservicios, y una variedad de métricas de evaluación, siendo las métricas de cohesión, IFN y LOC las más comunes. Ubicación: sección IV-B (Identificación, investigación y evaluación de microservicios), página 23393.

Nota personal:

Me ayuda a entender que la transformación de sistemas monolíticos a arquitecturas de microservicios es un proceso que presenta varias dificultades, y que depende críticamente de aplicar métricas de evaluación precisas (como cohesión y LOC) para separar correctamente los componentes. La usaré en TA-IND-01, en la sección Arquitectura del sistema, para fundamentar teóricamente cómo y con qué criterios está dividido Netflix.

3.2. Ficha 2: Descomposición de monolitos en microservicios

Referencia (IEEE):

Y. Abgaz et al., «Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,» *IEEE Transactions on Software Engineering*, vol. 49, n.º 8, págs. 4213-4242, ago. de 2023, ISSN: 2326-3881. DOI: 10.1109/tse.2023.3287297 dirección: http://dx.doi.org/10.1109/TSE.2023.3287297

DOI o ISBN verificado:

https://doi.org/10.1109/TSE.2023.3287297

Resumen:

Muchas empresas que utilizan sistemas complejos buscan soluciones para descomponer sus aplicaciones monolíticas en microservicios. Se examinaron 35 artículos de investigación mediante una revisión sistemática de la literatura. Los autores clasificaron los enfoques de descomposición según el tipo de dato en el que se basan: basados en modelos, en código, en registros y en versiones. Se observó una ausencia de una descripción del proceso de descomposición de extremo a extremo. Además, la descomposición de monolitos aún se encuentra en una etapa temprana, con falta de métodos y de herramientas de apoyo.

Dato concreto:

Los autores concluyen que las fases principales del M2MDF son: recopilación de datos de entrada, análisis de monolitos, identificación de microservicios, optimización, evaluación e implementación de microservicios. Ubicación: sección VIII (Conclusión), página 4236.

Nota personal:

Me ayuda a entender que hay varios enfoques para descomponer, no uno solo, y que aún faltan métodos y herramientas de apoyo. La usaré en TA-IND-01, en la sección Postura crítica, para argumentar sobre los desafíos técnicos que enfrentó la migración de Netflix a microservicios.

3.3. Ficha 3: Prácticas de seguridad en sistemas de microservicios

Referencia (IEEE):

A. Rezaei Nasab, M. Shahin, S. A. Hoseyni Raviz, P. Liang, A. Mashmool y V. Lenarduzzi, «An empirical study of security practices for microservices systems,» *Journal of Systems and Software*, vol. 198, pág. 111 563, abr. de 2023, ISSN: 0164-1212. DOI: 10.1016/j.jss.2022.111563 dirección: http://dx.doi.org/10.1016/j.jss.2022.111563

DOI o ISBN verificado:

https://doi.org/10.1016/j.jss.2022.111563

Resumen:

La seguridad en los sistemas de microservicios es un gran reto, principalmente por la brecha de conocimiento que existe entre los profesionales sobre cómo asegurarlos correctamente. Para ayudar con esto, el estudio extrajo información de repositorios y foros para proponer prácticas de seguridad en áreas clave como autorización, manejo de tokens y bases de datos. Como método de validación, en lugar de implementarlas en sistemas reales, realizaron una encuesta a 74 profesionales de la industria. Como resultado, los encuestados consideraron que estas prácticas eran útiles para su trabajo.

Dato concreto:

El API Gateway para sistemas grandes (PA4) fue la práctica más aceptada: 70,27 % la consideró "absolutamente útil" y 25,68 % "útil", o sea 95,95 % en total. Ubicación: tabla 3, sección 4.2.1.

Nota personal:

Me ayuda a entender que las prácticas de seguridad se organizan en varias categorías como autorización y tokens. La usaré en TA-IND-01, en la sección Postura crítica, para argumentar qué categorías debe cubrir Netflix.

3.4. Ficha 4: Herramientas de monitoreo para DevOps y microservicios

Referencia (IEEE):

L. Giamattei et al., «Monitoring tools for DevOps and microservices: A systematic grey literature review,» *Journal of Systems and Software*, vol. 208, pág. 111 906, feb. de 2024, ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111906 dirección: http://dx.doi.org/10.1016/j.jss.2023.111906

DOI o ISBN verificado:

https://doi.org/10.1016/j.jss.2023.111906

Resumen:

El desarrollo y operación de los sistemas basados en microservicios es una situación compleja. El principal problema que aborda el artículo es la dificultad de seleccionar una herramienta de monitorización adecuada para sistemas de microservicios desarrollados bajo prácticas DevOps. Se empleó una revisión sistemática de literatura gris, con un proceso riguroso de búsqueda, selección y extracción de datos en repositorios como GitHub y motores de búsqueda, que llevó a 71 herramientas para analizar. Como resultado, los autores contrastaron los resultados y sintetizaron un mapa de las características de las herramientas analizadas, con sus ventajas y desventajas.

Dato concreto:

La mayoría de las herramientas (58 %) están dirigidas a sistemas distribuidos de alto nivel, mientras que el resto, especialmente las más nuevas, se centran en tecnologías de servicios web (21 %) y en sistemas basados en microservicios (21 %). Ubicación: sección 8.1 (Principales hallazgos y orientaciones para ingenieros DevOps), página 13.

Nota personal:

Me ayuda a entender que monitorear es la base para detectar fallos, y que el principal problema es la dificultad de seleccionar una herramienta adecuada. La usaré en TA-IND-01, en el apartado de tolerancia a fallos, para explicar cómo Netflix logra detectar que un nodo o servicio dejó de responder.

3.5. Ficha 5: Visualización dinámica de arquitecturas de microservicios

Referencia (IEEE):

M. E. Gortney et al., «Visualizing Microservice Architecture in the Dynamic Perspective: A Systematic Mapping Study,» *IEEE Access*, vol. 10, págs. 119 999-120 012, 2022, ISSN: 2169-3536. DOI: 10.1109/access.2022.3221130 dirección: http://dx.doi.org/10.1109/ACCESS.2022.3221130

DOI o ISBN verificado:

https://doi.org/10.1109/ACCESS.2022.3221130

Resumen:

A medida que los microservicios van evolucionando y popularizándose, surgen diversos inconvenientes. Uno de estos es la falta de una visión completa del sistema, lo que impide visualizar la arquitectura de una forma integral. En este artículo se hizo un estudio de mapeo investigando el análisis dinámico para visualizar la arquitectura del sistema. Durante

el proceso de revisión literaria probaron con 37 consultas de búsquedas diferentes. Los autores identificaron tres técnicas principales de análisis dinámico: minería de procesos, rastreo distribuido y monitorización, además encontraron herramientas para recopilar datos de tiempo de ejecución y crear visualización a partir de su análisis.

Dato concreto:

Existen tres técnicas principales de análisis dinámico: minería de procesos, rastreo distribuido y monitorización. Ubicación: sección 9 (Conclusión), página 120011.

Nota personal:

Me ayuda a entender que la arquitectura se puede ver mientras el sistema se ejecuta, con tres técnicas: minería de procesos, rastreo distribuido y monitorización. La usaré en TA-IND-01 en la sección 4 (Arquitectura del sistema) para explicar cómo interactúan los componentes de Netflix.

4. Declaración de uso de inteligencia artificial

4.1. Castro Coello Mario Sebastián

Herramienta	**Versión**	**Para qué se usó**	**Cómo se verificó lo obtenido**
Claude Code	Claude Opus 5.5	Buscar fuentes candidatas en OpenAlex, comprobar sus datos en doi.org y consultar el cuartil de cada revista en SJR.	Abrí cada DOI, descargué el artículo y comparé autores, título, revista, volumen y páginas con la ficha. Elegí las cinco fuentes y reemplacé una de las propuestas.
Claude Code	Claude Opus 5.5	Agregar mis referencias al `.bib` con el script `doi2bib.ps1`, que descarga los datos oficiales de doi.org.	El script `verificar-referencias.ps1` confirmó que las cinco coinciden con doi.org, y las comparé con cada artículo.
Claude Code	Claude Opus 5.5	Armar la estructura base del repositorio grupal (un archivo de fichas, uno de referencias y uno de declaración por integrante) y crear las ramas de cada integrante.	Revisé la estructura con el grupo, compilé el documento y comprobé las ramas en GitHub.
Claude Code	Claude Opus 5.5	Revisar mis resúmenes, datos concretos y notas personales contra los artículos (número de palabras, páginas y secciones) y señalar imprecisiones.	Corregí yo cada punto señalado y volví a revisar los datos en cada artículo.
Claude Code	Claude Opus 5.5	Corregir el formato en LaTeX: que las referencias muestren todos los autores, que los enlaces no se salgan del margen, escapar caracteres especiales (`#`, `%`), cambiar comillas y quitar marcas de cita que no tenían referencia.	Compilé el documento, revisé que no hubiera desbordes ni citas sin resolver y que el texto de cada ficha se viera completo en el PDF.
Claude Code	Claude Opus 5.5	Revisar en una copia las ramas del grupo antes de fusionarlas (referencias, compilación, resúmenes repetidos y erratas) y, a pedido mío, fusionar en GitHub el PR de Elian y hacer el commit final con la fecha y el PDF regenerado.	Los scripts `verificar-referencias.ps1` y `compilar.ps1` confirmaron las referencias contra doi.org y la compilación sin desbordes. Cada integrante corrigió sus propias fichas.

4.2. Loor Suárez Jordan Josué

Herramienta	Versión	Para qué se usó	Cómo se verificó lo obtenido
ChatGPT	GPT-6	Orientación para comprender los artículos y localizar información relevante.	Lectura y comparación con los PDF originales.
ChatGPT	GPT-6	Apoyo para identificar problemas, métodos y resultados.	Revisión de las publicaciones originales.
ChatGPT	GPT-6	Orientación para localizar datos concretos y sus páginas.	Comprobación directa en los PDF.
ChatGPT	GPT-6	Apoyo técnico para organizar las fichas y utilizar LaTeX.	Revisión de los requisitos y del código.

4.3. Mendoza Villarroel Elian Jossue

Herramienta	Versión	Para qué se usó	Cómo se verificó lo obtenido
Claude	Opus 5.5, Sonnet 5.5	Buscar artículos candidatos.	Abrí cada DOI y comparé autores, año, título, revista, volumen y páginas.
Claude	Opus 5.5, Sonnet 5.5	Ubicar datos concretos en los artículos.	Los comprobé en el PDF de cada artículo (sección y página).
Claude	Opus 5.5, Sonnet 5.5	Explicar conceptos (DOI, biblatex, pull request).	Lo comprobé con las instrucciones de la actividad al aplicarlo en el repositorio.
Claude	Opus 5.5, Sonnet 5.5	Orientación sobre LaTeX, BibTeX y GitHub.	Obtuve el BibTeX desde doi.org y probé los pasos en GitHub.
Claude	Opus 5.5, Sonnet 5.5	Revisar ortografía y redacción de mis textos.	Leí cada corrección y la acepté o la cambié yo; las ideas son mías.
Claude	Opus 5.5, Sonnet 5.5	Contar palabras de los resúmenes y detectar errores de formato en mi archivo .tex.	Comprobé los resultados en mi propio archivo.
Claude	Opus 5.5, Sonnet 5.5	Sugerir ideas y estructura para las notas personales.	Reescribí las notas con mis palabras y comprobé que lo que afirmo del sistema fuera cierto.

Referencias

[1] V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,» *IEEE Access*, vol. 11, págs. 88 339-88 358, 2023. DOI: 10.1109/ACCESS.2023.3305687

[2] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Benchmarking Consistency Levels of Cloud-Distributed NoSQL Databases Using YCSB,» *IEEE Access*, vol. 13, págs. 63 428-63 438, 2025. DOI: 10.1109/ACCESS.2025.3558923

[3] A. U. Rehman, R. L. Aguiar y J. P. Barraca, «Fault-Tolerance in the Scope of Cloud Computing,» *IEEE Access*, vol. 10, págs. 63 422-63 441, 2022. DOI: 10.1109/ACCESS.2022.3182211

[4] W. Cazzola, F. Cesarini y L. Tansini, «PerformERL: a performance testing framework for erlang,» *Distributed Computing*, vol. 35, n.º 5, págs. 439-454, 2022. DOI: 10.1007/s00446-022-00429-7

[5] G. Blinowski, A. Ojdowska y A. Przybylek, «Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,» *IEEE Access*, vol. 10, págs. 20 357-20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

[6] R. Kanishka Jayalath, H. Ahmad, D. Goel, M. Shuja Syed y F. Ullah, «Microservice Vulnerability Analysis: A Literature Review With Empirical Insights,» *IEEE Access*, vol. 12, págs. 155 168-155 204, 2024. DOI: 10.1109/ACCESS.2024.3481374

[7] A. Lercher, J. Glock, C. Macho y M. Pinzger, «Microservice API Evolution in Practice: A Study on Strategies and Challenges,» *Journal of Systems and Software*, vol. 215, 2024, **artno** 112110. DOI: 10.1016/j.jss.2024.112110

[8] M. Franco, O. Gaggi, B. Guidi, A. Michienzi y C. E. Palazzi, «A Decentralised Messaging System Robust Against the Unauthorised Forwarding of Private Content,» *Future Generation Computer Systems*, vol. 145, págs. 211-222, 2023. DOI: 10.1016/j.future.2023.03.025

[9] P. d. F. Melo, M. Hoseini, S. Zannettou y F. Benevenuto, «Don't Break the Chain: Measuring Message Forwarding on WhatsApp,» en *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 18, 2024, págs. 1054-1067. DOI: 10.1609/icwsm.v18i1.31372

[10] C. Colarusso, A. De Caro, I. Falco, L. Goglia y E. Zimeo, «A distributed tracing pipeline for improving locality awareness of microservices applications,» *Software: Practice and Experience*, vol. 54, n.º 6, págs. 1118-1140, 2024. DOI: 10.1002/spe.3317

[11] I. Oumoussa y R. Saidi, «Evolution of Microservices Identification in Monolith Decomposition: A Systematic Review,» *IEEE Access*, vol. 12, págs. 23 389-23 405, 2024, ISSN: 2169-3536. DOI: 10.1109/access.2024.3365079 dirección: http://dx.doi.org/10.1109/ACCESS.2024.3365079

[12] Y. Abgaz et al., «Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review,» *IEEE Transactions on Software Engineering*, vol. 49, n.º 8, págs. 4213-4242, ago. de 2023, ISSN: 2326-3881. DOI: 10.1109/tse.2023.3287297 dirección: http://dx.doi.org/10.1109/TSE.2023.3287297

[13] A. Rezaei Nasab, M. Shahin, S. A. Hoseyni Raviz, P. Liang, A. Mashmool y V. Lenarduzzi, «An empirical study of security practices for microservices systems,» *Journal of Systems and Software*, vol. 198, pág. 111 563, abr. de 2023, ISSN: 0164-1212. DOI: 10.1016/j.jss.2022.111563 dirección: http://dx.doi.org/10.1016/j.jss.2022.111563

[14] L. Giamattei et al., «Monitoring tools for DevOps and microservices: A systematic grey literature review,» *Journal of Systems and Software*, vol. 208, pág. 111 906, feb. de 2024, ISSN: 0164-1212. DOI: 10.1016/j.jss.2023.111906 dirección: http://dx.doi.org/10.1016/j.jss.2023.111906

[15] M. E. Gortney et al., «Visualizing Microservice Architecture in the Dynamic Perspective: A Systematic Mapping Study,» *IEEE Access*, vol. 10, págs. 119 999-120 012, 2022, ISSN: 2169-3536. DOI: 10.1109/access.2022.3221130 dirección: http://dx.doi.org/10.1109/ACCESS.2022.3221130